תקציר השיעור: דיפ לרנינג, RNN ו-LSTM

השיעור התמקד ביישור קו והעמקת ההבנה בנושא **Deep Learning**, עם דגש מיוחד על **רשתות נוירונים רקורנטיות (RNN)** ורשתות **Long Short-Term Memory (LSTM)**, כולל הקשר שלהן למודלים כגון **טרנספורמרים**. המטרה המרכזית הייתה לפתח תחושה והבנה **"מבפנים"** של המודלים, במקום רק ידע טכני.

הנחיות ודרישות קורס

1. ציפיות מהסטודנטים

Yoram מדגיש כי המטרה היא להוביל לרמה גבוהה של **הובלה וחקר**.

- **הבנה מעמיקה:** נדרשת הבנה של הרעיון והתחושה הפנימית של המודלים, ולא רק היכולת הטכנית להשתמש בהם.
- **חקר וניתוח תוצאות:** יש לנתח את תוצאות המודלים (לדוגמה, במטלה הראשונה), לבדוק את מגבלותיהם ולציין במה הם טובים ובמה פחות.
- **ניהול כלי AI:** הציפייה היא לראות כיצד הסטודנטים **מנהלים** את כלי הבינה המלאכותית (כגון Claude CLI), ולא נותנים לכלי לנהל אותם. יש להפגין כושר ביטוי, הבנה, ויכולת להתמודד עם באגים ולכוון את הכלי.

2. דרישות חומרה ותוכנה

- **סביבת עבודה מומלצת:** מומלץ לעבוד עם כלי **Claude CLI** (או כלים דומים כגון Gemini CLI או Codex).
  - **Claude CLI:** הוא Virtual Machine המאפשר ל-LLM לבצע פעולות (כתיבה, עריכה, הפעלת קוד) בתוך תיקייה ספציפית במחשב, תוך בקשת רשות על כל פעולה לצרכי אבטחה.
  - **יתרון:** מאפשר **פיתוח קוד זריז (Live Coding)** ללא צורך במעבר קופי-פייסט בין VSCode/IDE ל-AI.
- **כלים נוספים (רשות):** ניתן להשתמש ב-Jupyter, Discourse ובספריות ML רלוונטיות.

3. הנחיות כלליות למטלות

- **ציון עצמי:** כל סטודנט/קבוצה נדרש לתת **ציון עצמי** למטלה. הציון מכתיב את רמת הבדיקה:
  - **ציון 100:** מחייב הגשה מושלמת ללא שום שגיאות (כולל שגיאות כתיב או הדפסה).
  - **ציון 60:** מאפשר **בדיקה מכילה** (בדיקה שבה הוגש חומר רלוונטי, בכיוון, ועם היגיון).
- **הגשה (כללית):** ההגשה היא כקובץ **PDF** ב-Moodle, המכיל לינק ל- **GitHub Repository** גלובלי.
- **מבנה קוד מומלץ:** יש לשאוף לקבצים קטנים (עד 150 שורות לקובץ), עם הסברים קצרים ובהירים בתוך הקוד.
- **שיתוף:** העבודות נשמרות באופן גלובלי (פומבי) על מנת לאפשר למידה הדדית.

4. מבנה דוח הגשה (כללי)

הגשת העבודה צריכה לשקף פרויקט תוכנה רציני. **המינימום הנדרש** לדוח ההגשה הוא הצגת תוצאות גרפיות וניתוח השגיאה:

- **Git Readme:** קובץ Readme ב-GitHub שישמש כדוח.
- **צילומי מסך:** צילומי מסך (של גרפים, קטעי קוד רלוונטיים, ואינטראקציה עם כלי ה-AI) ב-Readme.

🧠 מושגי יסוד ורשתות נוירונים

1. 📐 וקטורים ומדד דמיון

- **הגדרת וקטור:** וקטור הוא סדרה של מספרים, ייצוג של נקודה במרחב רב-ממדי, ובעל גודל וכיוון.
- **העוצמה בעולם AI:** עוצמתו של הווקטור באה לידי ביטוי ביכולתו לשמש **מדד לדמיון** בין אובייקטים/נתונים שונים.
- **מטרת AI:** המטרה היא לגלות את ה"אל" (החוקיות) מתוך ה"יש" (הדאטה), וזה נעשה על ידי מציאת דומים באמצעות וקטורים.

דוגמת המרצה (משתמעת): ילד וחתול

כשאנחנו רוצים ללמד רשת לזהות ילד או חתול, אנחנו ממירים את התמונה (ה"יש") לווקטור פיצ'רים (לדוגמה: וקטור שמתאר גובה, משקל, צבע, וכו'). המודל מחפש וקטורים דומים כדי לדעת למי שייך הנתון החדש.

2. 🧠 פרספטרון ורשתות Fully Connected (FC)

- **פרספטרון כקו ישר:** הוכח מתמטית כי פעולת הלימוד הבסיסית של פרספטרון בודד ($vec{W} \cdot \vec{X} + B\$) מקבילה למציאת משוואת קו ישר (או מישור/היפר-מישור) המפריד בין קבוצות.
  - **הפרדה:** אם התוצאה חיובית (מעל הקו) שייך לקבוצה אחת, ואם שלילית (מתחת לקו) שייך לקבוצה אחרת.
- **שכבת FC כהפרדה חתיכות (Piecewise):** שכבת נוירונים (FC) מאפשרת לפתור בעיות שאינן ליניאריות (כאשר לא ניתן להפריד את הקבוצות בקו ישר אחד), על ידי יצירת מספר **קווי הפרדה** שונים (Piecewise).
  - **פעולת ה-FC:** ה-FC יוצרת וקטור מוצא שכל רכיב בו מייצג חתיכת הפרדה אחרת.
  - **פונקציית האקטיבציה:** מחליטה האם נקודה שייכת לאזור הפרדה מסוים (חיובי/שלילי).
- **החשיבות של FC:** עם כוח חישוב ודאטה אינסופיים, רשת FC יכולה לפתור **כל בעיה וכל פונקציה לא-ליניארית**. לכן, שכבת FC נדחפת לעיתים קרובות בסוף ארכיטקטורות (כמו ResNet) כדי "לסגור את הפינות" ולטפל בבעיות שנותרו.

3. 🔄 Recurrent Neural Networks (RNN)

- **עבודה עם סדרות:** RNN מיועדת לעבודה עם **סדרות** (משפטים, סדרות עיתיות, אודיו, וידאו, נתוני IoT), כלומר מידע שיש בו **סדר**.
  - **מטרה:** לנצל את **המידע הטמפורלי (הזמני)** כדי להקל על החיזוי/החישוב.
- **מנגנון הזיכרון (Memory):** החידוש ב-RNN הוא אלמנט הזיכרון (המכונה) המאפשר לה להשתמש בתוצאה (התחזית) של הצעד הקודם ($text{W}_{H} \cdot H_{t-1}\$) כקלט בצעד הנוכחי.
  - **יתרון:** מאפשרת טיפול בבעיות סדרתיות עם **מעט משקלים** (Weights) קבועים.
- **סוגי מידע בזמן:**
  - **תדר גבוה (שינויים קרובים):** מידע על שינויים בטווחי זמן קצרים (מה קרה לפני דקה).
  - **תדר נמוך (שינויים רחוקים):** מידע על מגמות ושינויים בטווחי זמן ארוכים (מה קרה לפני חודש, **טרנדים**).
  - **משקולות ודגש:** הרשת לומדת לתת משקלים גבוהים למידע רחוק (כדי לגלות טרנדים) או למידע קרוב (כדי לגלות שינויים מהירים).
- **התמודדות עם אורך רצף (זיכרון ארוך):** כדי להסתכל על יותר מצעד אחד אחורה, יש צורך **לשכפל את יחידת הזיכרון** (Memory Unit) ולהכניס אותה כקלט נוסף.

4. ⏳ Long Short-Term Memory (LSTM)

- **הבעיה ב-RNN:** ב-RNN, הריצה במעגל ופעולת המכפלה החוזרת ב- Backpropagation through Time גורמת לבעיית **היעלמות או התפוצצות הגרדיאנטים** (Vanishing/Exploding Gradients), מה שהורג את יכולת הרשת ללמוד תלויות ארוכות טווח.
- **פתרון LSTM:** LSTM היא ארכיטקטורת תאים שפותחה כדי **לשמור על המשקלים והגרדיאנטים לאורך זמן**.
  - **מבנה:** ה-LSTM מורכב מ-4 פרספטרונים בסיסיים, המכונים **שערים (Gates)**.
  - **מנגנון השערים:** השערים (Forget, Input, Output) מבצעים מניפולציות (כפל וחיבור) על ערכי הזיכרון, במטרה **לחזק או להנמיך** ערכים (כמו ב-Attention):
    - **שער Forget:** אחראי על **שכחה** של מידע ישן (מוריד ערכים).
    - **שער Input:** אחראי על **למידה** של מידע חדש (מעלה ערכים).
    - **שער Output/Cell State:** אחראי על הזיכרון לטווח **ארוך** (Long-Term Memory), שמאפשר לראות דברים רחוקים.

🏠 מטלת בית: חילוץ תדרים באמצעות LSTM

המטלה נועדה להמחיש את יכולת ה-LSTM לפעול כ**פילטר תדרים** ולבצע חילוץ מידע ספציפי מתוך אותות מעורבבים.

ראו פרוט מדוייק במקום הגשת המטלה

כאן יש סרטון מיוטיוב המסביר את המושגים המרכזיים שנדונו בשיעור, כגון RNN ו-LSTM:
[Recurrent Neural Networks (RNNs), Clearly Explained!!!](#).